

LAB PROGRAM 1

QUESTION:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)

CODE:

a)FCFS:

```
#include <stdio.h>

int main() {
    int n, bt[20], wt[20], tat[20], at[20], ct[20];
    int i;
    float twt = 0.0, ttat = 0.0, awt, att;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    // Input Arrival Time and Burst Time
    for (i = 0; i < n; i++) {
        printf("Enter Arrival Time for Process %d: ", i + 1);
        scanf("%d", &at[i]);
        printf("Enter Burst Time for Process %d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // FCFS Scheduling
    ct[0] = at[0] + bt[0]; // Completion time of first process
```

```
wt[0] = 0; // Waiting time of first process
```

```
tat[0] = bt[0]; // Turnaround time
```

```
for (i = 1; i < n; i++) {
```

```
if (ct[i - 1] < at[i]) {
```

```
ct[i] = at[i] + bt[i]; // CPU idle
```

```
wt[i] = 0;
```

```
} else {
```

```
wt[i] = ct[i - 1] - at[i];
```

```
ct[i] = ct[i - 1] + bt[i];
```

```
}
```

```
tat[i] = wt[i] + bt[i];
```

```
}
```

```
// Calculate averages
```

```
for (i = 0; i < n; i++) {
```

```
twt += wt[i];
```

```
ttat += tat[i];
```

```
}
```

```
awt = twt / n;
```

```
att = ttat / n;
```

```
// Output Table
```

```
printf("\nPROCESS\tAT\tBT\tCT\tWT\tTAT");
```

```
for (i = 0; i < n; i++) {
```

```
printf("\nP%d\t%d\t%d\t%d\t%d\t",
```

```

i + 1, at[i], bt[i], ct[i], wt[i], tat[i]);
}

printf("\n\nAverage Waiting Time = %.2f", awt);
printf("\n\nAverage Turnaround Time = %.2f\n", att);

return 0;
}

```

RESULT:

```

PS D:\Studies\OS\lab\lab programs\lp1> & "D:\Studies\OS\lab\lab programs\lp1\FCFS.
exe"
Enter the number of processes: 4
Enter Arrival Time for Process 1: 0
Enter Burst Time for Process 1: 7
Enter Arrival Time for Process 2: 3
Enter Burst Time for Process 2: 4
Enter Arrival Time for Process 3: 6
Enter Burst Time for Process 3: 6
Enter Arrival Time for Process 4: 8
Enter Burst Time for Process 4: 3

PROCESS AT      BT      CT      WT      TAT
P1        0       7       7       0       7
P2        3       4      11       4       8
P3        6       6      17       5      11
P4        8       3      20       9      12

Average Waiting Time = 4.50
Average Turnaround Time = 9.50
PS D:\Studies\OS\lab\lab programs\lp1> 

```

b) 1)SJF:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, bt[20], wt[20], tat[20], at[20], ct[20];
```

```
    int completed[20] = {0};
```

```
    int i, time = 0, count = 0, min, index;
```

```
    float twt = 0.0, ttat = 0.0, awt, att;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time for Process %d: ", i + 1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("Enter Burst Time for Process %d: ", i + 1);
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    while (count < n) {
```

```
        min = 9999;
```

```
        index = -1;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (at[i] <= time && completed[i] == 0) {
```

```
                if (bt[i] < min) {
```

```
                    min = bt[i];
```



```

        index = i;
    }
}
}

if (index != -1) {
    time += bt[index];
    ct[index] = time;
    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];
    completed[index] = 1;
    count++;
} else {
    time++; // CPU idle
}
}

for (i = 0; i < n; i++) {
    twt += wt[i];
    ttat += tat[i];
}

awt = twt / n;
att = ttat / n;

printf("\nPROCESS\tAT\tBT\tCT\tWT\tTAT");
for (i = 0; i < n; i++) {
    printf("\nP%d\t%d\t%d\t%d\t%d\t%d",

```

```

        i + 1, at[i], bt[i], ct[i], wt[i], tat[i]);
    }

    printf("\n\nAverage Waiting Time = %.2f", awt);
    printf("\n\nAverage Turnaround Time = %.2f\n", att);

    return 0;
}

```

RESULT:

```

PS D:\Studies\OS\lab\lab programs> & "D:\Studies\OS\lab\lab programs\lp1\SJF.exe"
Enter the number of processes: 4
Enter Arrival Time for Process 1: 0
Enter Burst Time for Process 1: 7
Enter Arrival Time for Process 2: 3
Enter Burst Time for Process 2: 4
Enter Arrival Time for Process 3: 6
Enter Burst Time for Process 3: 6
Enter Arrival Time for Process 4: 8
Enter Burst Time for Process 4: 3

PROCESS AT      BT      CT      WT      TAT
P1      0       7       7       0       7
P2      3       4      11       4       8
P3      6       6      20       8      14
P4      8       3      14       3       6

Average Waiting Time = 3.75
Average Turnaround Time = 8.75

```

2)SRTF:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, bt[20], wt[20], tat[20], at[20], ct[20], rt[20];
```

```
    int i, time = 0, count = 0, min, index;
```

```
    float twt = 0.0, ttat = 0.0, awt, att;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time for Process %d: ", i + 1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("Enter Burst Time for Process %d: ", i + 1);
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i]; // Remaining time
```

```
    }
```

```
    while (count < n) {
```

```
        min = 9999;
```

```
        index = -1;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (at[i] <= time && rt[i] > 0) {
```

```
                if (rt[i] < min) {
```

```
                    min = rt[i];
```

```
                    index = i;
```

```

    }

}

}

if (index != -1) {
    rt[index]--; // Execute for 1 unit
    time++;

    if (rt[index] == 0) {
        ct[index] = time;
        tat[index] = ct[index] - at[index];
        wt[index] = tat[index] - bt[index];
        count++;
    }
} else {
    time++; // CPU idle
}

}

for (i = 0; i < n; i++) {
    twt += wt[i];
    ttat += tat[i];
}

awt = twt / n;
att = ttat / n;

printf("\nPROCESS\tAT\tBT\tCT\tWT\tTAT");

```

```

for (i = 0; i < n; i++) {

    printf("\nP%d\t%d\t%d\t%d\t%d\t%d",

        i + 1, at[i], bt[i], ct[i], wt[i], tat[i]);

}

printf("\n\nAverage Waiting Time = %.2f", awt);

printf("\n\nAverage Turnaround Time = %.2f\n", att);

return 0;

}

```

RESULT:

```

● PS D:\Studies\OS\lab\lab programs\lp1> & "D:\Studies\OS\lab\lab programs\lp1\SRTF.
exe"
Enter the number of processes: 4
Enter Arrival Time for Process 1: 00
Enter Burst Time for Process 1: 8
Enter Arrival Time for Process 2: 1
Enter Burst Time for Process 2: 4
Enter Arrival Time for Process 3: 2
Enter Burst Time for Process 3: 2
Enter Arrival Time for Process 4: 3
Enter Burst Time for Process 4: 1

PROCESS AT      BT      CT      WT      TAT
P1      0      8      15      7      15
P2      1      4      8      3      7
P3      2      2      4      0      2
P4      3      1      5      1      2

Average Waiting Time = 2.75
Average Turnaround Time = 6.50

```

c)PRIORITY:

1)NON PREEMPTIVE:

```
#include<stdio.h>
```

```
struct process{
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int pr;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
    int rt;
```

```
    int done;
```

```
};
```

```
float average(struct process p[],int n,int choice) {
```

```
    int sum=0;
```

```
    for(int i=0;i<n;i++) {
```

```
        if(choice==1)
```

```
            sum+=p[i].tat;
```

```
        else if(choice==2)
```

```
            sum+=p[i].wt;
```

```
    }
```

```
    return (float)sum/n;
```

```
}
```

```

void calcCT(struct process p[],int n) {

    int completed=0;
    int time=0;

    while(completed<n) {

        int idx=-1;
        int bestPriority=999;
        //running a loop to select the process arrived with highest priority
        for(int i=0;i<n;i++) {

            if(p[i].at<=time && p[i].done==0) {

                if(p[i].pr < bestPriority) {
                    bestPriority=p[i].pr;
                    idx=i;
                }

            }
        }

        // CPU idle
        if(idx== -1) {
            time++;
        }

        //logic for calculating ct for selected highest priority arrived process
    }
}

```

```

    else {
        time += p[idx].bt;
        p[idx].ct = time;
        p[idx].done = 1;
        completed++;
    }
}
}

```

```

void calcTAT(struct process p[],int n) {
    for(int i=0;i<n;i++) {
        p[i].tat = p[i].ct - p[i].at;
    }
}

```

```

void calcWT(struct process p[],int n) {
    for(int i=0;i<n;i++) {
        p[i].wt = p[i].tat - p[i].bt;
        p[i].rt = p[i].wt;
    }
}

```

```

int main() {

    struct process p[5]={
        {1,0,7,3,0,0,0,0,0},
        {2,2,4,1,0,0,0,0,0},
        {3,3,6,4,0,0,0,0,0},

```



```

        {4,5,5,2,0,0,0,0,0},
        {5,6,2,1,0,0,0,0,0}
    };

    int n=5;

    calcCT(p,n);
    calcTAT(p,n);
    calcWT(p,n);

    printf("PROCESS\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n\n");

    for(int i=0;i<n;i++) {

        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
            p[i].at,
            p[i].bt,
            p[i].pr,
            p[i].ct,
            p[i].tat,
            p[i].wt,
            p[i].rt);
    }

    printf("\nAverage Turnaround Time = %.2f", average(p,n,1));
    printf("\nAverage Waiting Time = %.2f\n", average(p,n,2));
}

```

2)PREEMPTIVE:

```
#include<stdio.h>
```

```
struct process{
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int pr;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
    int rt;
```

```
};
```

```
float average(struct process p[],int n,int choice) {
```

```
    int sum=0;
```

```
    for(int i=0;i<n;i++) {
```

```
        if(choice==1)
```

```
            sum+=p[i].tat;
```

```
        else
```

```
            sum+=p[i].wt;
```

```
    }
```

```
    return (float)sum/n;
```

```
}
```

```
void calcCT(struct process p[],int n) {
```

```
    int completed=0;
```

```
    int time=0;
```

```
    while(completed<n) {
```

```
        int idx=-1;
```

```
        int bestPriority=999;
```

```
        for(int i=0;i<n;i++) {
```

```
            if(p[i].at<=time && p[i].rt>0) {
```

```
                if(p[i].pr < bestPriority) {
```

```
                    bestPriority=p[i].pr;
```

```
                    idx=i;
```

```
                }
```

```
            }
```

```
        }
```

```
        if(idx!=-1) {
```

```
            time++;
```

```
        }
```

```
    else {
```

```
        p[idx].rt--; // run for 1 unit
```

```

        time++;

        if(p[idx].rt==0) {
            p[idx].ct=time;
            completed++;
        }
    }
}

```

```

void calcTAT(struct process p[],int n) {
    for(int i=0;i<n;i++) {
        p[i].tat=p[i].ct-p[i].at;
    }
}

```

```

void calcWT(struct process p[],int n) {
    for(int i=0;i<n;i++) {
        p[i].wt=p[i].tat-p[i].bt;
    }
}

```

```

int main() {

    struct process p[5]={
        {1,0,7,3,0,0,0,7},
        {2,2,4,1,0,0,0,4},
        {3,3,6,4,0,0,0,6},

```

```

        {4,5,5,2,0,0,0,5},
        {5,6,2,1,0,0,0,2}
    };

    int n=5;

    calcCT(p,n);
    calcTAT(p,n);
    calcWT(p,n);

    printf("PROCESS\tAT\tBT\tPR\tCT\tTAT\tWT\n\n");

    for(int i=0;i<n;i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
            p[i].at,
            p[i].bt,
            p[i].pr,
            p[i].ct,
            p[i].tat,
            p[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", average(p,n,1));
    printf("\nAverage Waiting Time = %.2f\n", average(p,n,2));

}

```

RESULT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\1bf24cs289\os lab> cd "d:\1bf24cs289\os lab\lab2\" ; if ($?) { gcc priority_nonpre.c -o pr
iority_nonpre } ; if ($?) { .\priority_nonpre }
PROCESS AT BT PR CT TAT WT RT
1 0 7 3 7 7 0 0
2 2 4 1 11 9 5 5
3 3 6 4 24 21 15 15
4 5 5 2 18 13 8 8
5 6 2 1 13 7 5 5

Average Turnaround Time = 11.40
Average Waiting Time = 6.60
PS D:\1bf24cs289\os lab\lab2> cd "d:\1bf24cs289\os lab\lab2\" ; if ($?) { gcc priority_pre.c -o
priority_pre } ; if ($?) { .\priority_pre }
PROCESS AT BT PR CT TAT WT
1 0 7 3 18 18 11
2 2 4 1 6 4 0
3 3 6 4 24 21 15
4 5 5 2 13 8 3
5 6 2 1 8 2 0

Average Turnaround Time = 10.60
Average Waiting Time = 5.80
PS D:\1bf24cs289\os lab\lab2> 
```

d)ROUND ROBIN:

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
struct Process {
```

```
    int pid;    // Process ID
```

```
    int at;     // Arrival Time
```

```
    int bt;     // Burst Time
```

```
    int rbt;    // Remaining Burst Time
```

```
    int ct;     // Completion Time
```

```
    int tat;    // Turnaround Time
```

```
    int wt;     // Waiting Time
```

```
    int rt;     // Response Time
```

```
    bool first_time;// Flag to check if process is getting CPU for the first time  
};
```

```
int main() {
```

```
    int n, tq;
```

```
    float total_tat = 0, total_wt = 0;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    // 1. Take Inputs
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Enter PID, Arrival Time, and Burst Time for process %d: ", i + 1);
```

```
        scanf("%d %d %d", &p[i].pid, &p[i].at, &p[i].bt);
```

```
        p[i].rbt = p[i].bt;
```

```
        p[i].first_time = true;
```

```
    }
```

```
    printf("Enter Time Quantum: ");
```

```
    scanf("%d", &tq);
```

```
    // 2. Sort processes by Arrival Time (Bubble Sort for simplicity)
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
```

```
            if (p[j].at > p[j+1].at) {
```

```
                struct Process temp = p[j];
```

```

        p[j] = p[j+1];
        p[j+1] = temp;
    }
}
}

```

// 3. Initialize Ready Queue

```

int queue[1000];
int front = 0, rear = 0;

int current_time = 0;
int completed = 0;
int i = 0; // Tracks the next process to arrive

```

// Fast-forward time to the first process's arrival if CPU is idle at t=0

```

if (p[0].at > current_time) {
    current_time = p[0].at;
}

```

// Enqueue all processes that have arrived at the starting time

```

while (i < n && p[i].at <= current_time) {
    queue[rear++] = i;
    i++;
}

```

// 4. Execute Round Robin Simulation

```

while (completed < n) {
    // If queue is empty but processes are pending, fast-forward time to next arrival

```



```
if (front == rear) {  
    current_time = p[i].at;  
    while (i < n && p[i].at <= current_time) {  
        queue[rear++] = i;  
        i++;  
    }  
}
```

```
// Dequeue process
```

```
int idx = queue[front++];
```

```
// Calculate Response Time if getting CPU for the first time
```

```
if (p[idx].first_time) {  
    p[idx].rt = current_time - p[idx].at;  
    p[idx].first_time = false;  
}
```

```
// Execute process for Time Quantum or Remaining Burst Time, whichever is smaller
```

```
int exec_time = (p[idx].rbt < tq) ? p[idx].rbt : tq;
```

```
current_time += exec_time;
```

```
p[idx].rbt -= exec_time;
```

```
// Enqueue newly arrived processes DURING this execution window
```

```
while (i < n && p[i].at <= current_time) {  
    queue[rear++] = i;  
    i++;  
}
```

```

// If the current process is not finished, put it back at the end of the queue
if (p[idx].rbt > 0) {
    queue[rear++] = idx;
} else {
    // Process is finished: Calculate Completion, Turnaround, and Waiting Times
    p[idx].ct = current_time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;

    total_tat += p[idx].tat;
    total_wt += p[idx].wt;
    completed++;
}
}

```

// 5. Sort back by PID to print results neatly in their original order

```

for (int j = 0; j < n - 1; j++) {
    for (int k = 0; k < n - j - 1; k++) {
        if (p[k].pid > p[k+1].pid) {
            struct Process temp = p[k];
            p[k] = p[k+1];
            p[k+1] = temp;
        }
    }
}

```

// 6. Print the Table

```

printf("\n-----");

```

```

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

printf("-----\n");

for (int j = 0; j < n; j++) {

    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",

           p[j].pid, p[j].at, p[j].bt, p[j].ct, p[j].tat, p[j].wt, p[j].rt);

}

printf("-----\n");

printf("Average Turnaround Time: %.2f\n", total_tat / n);

printf("Average Waiting Time: %.2f\n", total_wt / n);

return 0;

}

```

RESULT:

```

PS D:\1bf24cs289> cd "d:\1bf24cs289\" ; if ($?) { gcc roundRobin.c -o roundRobin } ; if ($?) { .\roundRobin }
Enter the number of processes: 5
Enter PID, Arrival Time, and Burst Time for process 1: 1 0 4
Enter PID, Arrival Time, and Burst Time for process 2: 2 1 2
Enter PID, Arrival Time, and Burst Time for process 3: 3 2 3
Enter PID, Arrival Time, and Burst Time for process 4: 4 2 2
Enter PID, Arrival Time, and Burst Time for process 5: 5 8 5
Enter Time Quantum: 3

-----
PID    AT    BT    CT    TAT    WT    RT
-----
1      0     4    11    11     7     0
2      1     2     5     4     2     2
3      2     3     8     6     3     3
4      2     2    10     8     6     6
5      8     5    16     8     3     3
-----

Average Turnaround Time: 7.40
Average Waiting Time: 4.20

```

```

PS D:\1bf24cs289> cd "d:\1bf24cs289\" ; if ($?) { gcc roundRobin.c -o roundRobin } ; if ($?) { .\roundRobin }
Enter the number of processes: 5
Enter PID, Arrival Time, and Burst Time for process 1: 1 0 4
Enter PID, Arrival Time, and Burst Time for process 2: 2 1 2
Enter PID, Arrival Time, and Burst Time for process 3: 3 2 3
Enter PID, Arrival Time, and Burst Time for process 4: 4 2 2
Enter PID, Arrival Time, and Burst Time for process 5: 5 8 5
Enter Time Quantum: 6

-----
PID    AT    BT    CT    TAT    WT    RT
-----
1      0     4     4     4     0     0
2      1     2     6     5     3     3
3      2     3     9     7     4     4
4      2     2    11     9     7     7
5      8     5    16     8     3     3
-----

Average Turnaround Time: 6.60
Average Waiting Time: 3.40
PS D:\1bf24cs289> 

```

LAB PROGRAM 2

QUESTION:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

1)MULTI LEVEL NON PREEMPTIVE

```
//multilevel queue scheduling along with gantt chart
```

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int at; // Arrival Time
```

```
    int bt; // Burst Time
```

```
    int ct; // Completion Time
```

```
    int wt; // Waiting Time
```

```
    int tat; // Turnaround Time
```

```
    int type; // 0 = System, 1 = User
```

```
    int done; // 0 = not done, 1 = completed
```

```
};
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0;
```

```
    printf("Enter total number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```

// Input
for(i = 0; i < n; i++) {
    printf("\nProcess %d\n", i+1);

    p[i].pid = i + 1;

    printf("Enter Arrival Time: ");
    scanf("%d", &p[i].at);

    printf("Enter Burst Time: ");
    scanf("%d", &p[i].bt);

    printf("Enter Type (0 = System, 1 = User): ");
    scanf("%d", &p[i].type);

    p[i].done = 0;
}

// Scheduling
while(completed < n) {
    int idx = -1;

    // Step 1: Check System processes first
    for(i = 0; i < n; i++) {
        if(p[i].at <= time && p[i].done == 0 && p[i].type == 0) {
            idx = i;
            break; // FCFS → first arrived
        }
    }
}

```

```
}
```

```
// Step 2: If no system process, check user
```

```
if(idx == -1) {
```

```
    for(i = 0; i < n; i++) {
```

```
        if(p[i].at <= time && p[i].done == 0 && p[i].type == 1) {
```

```
            idx = i;
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
// Step 3: If no process available → increment time
```

```
if(idx == -1) {
```

```
    time++;
```

```
    continue;
```

```
}
```

```
// Execute process
```

```
time += p[idx].bt;
```

```
p[idx].ct = time;
```

```
p[idx].tat = p[idx].ct - p[idx].at;
```

```
p[idx].wt = p[idx].tat - p[idx].bt;
```

```
p[idx].done = 1;
```

```
completed++;
```

```
}
```

```
// Output
```

```
float avg_wt = 0, avg_tat = 0;
```

```
printf("\nPID\tAT\tBT\tType\tCT\tTAT\tWT\n");
```

```
for(i = 0; i < n; i++) {
```

```
    printf("%d\t%d\t%d\t%s\t%d\t%d\t%d\n",
```

```
        p[i].pid, p[i].at, p[i].bt,
```

```
        (p[i].type == 0) ? "SYS" : "USR",
```

```
        p[i].ct, p[i].tat, p[i].wt);
```

```
    avg_wt += p[i].wt;
```

```
    avg_tat += p[i].tat;
```

```
}
```

```
printf("\nAverage WT = %.2f", avg_wt / n);
```

```
printf("\nAverage TAT = %.2f\n", avg_tat / n);
```

```
//displaying gantt chart
```

```
for(i = 0; i < n; i++) {
```

```
    printf(" | P%d ", p[i].pid);
```

```
}
```

```
printf("\n");
```

```
for(i = 0; i < n; i++) {
```

```
    printf("%d  ", p[i].ct);
```

```
}
```

```
printf("\n");
```

```

    return 0;
}

```

RESULT:

```

PS D:\1bf24cs289> cd "d:\1bf24cs289\" ; if ($?) { gcc multilevel.c -o multilevel } ; if ($?) { .\multilevel }
● Enter total number of processes: 4

Process 1
Enter Arrival Time: 0
Enter Burst Time: 5
Enter Type (0 = System, 1 = User): 1

Process 2
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Type (0 = System, 1 = User): 0

Process 3
Enter Arrival Time: 2
Enter Burst Time: 4
Enter Type (0 = System, 1 = User): 1

Process 4
Enter Arrival Time: 3
Enter Burst Time: 2
Enter Type (0 = System, 1 = User): 0

PID    AT    BT    Type  CT    TAT   WT
1       0     5     USR    5     5     0
2       1     3     SYS    8     7     4
3       2     4     USR   14     8     8
4       3     2     SYS   10     7     5

Average WT = 4.25
Average TAT = 7.75
| P1 | P2 | P3 | P4 |
5    8   14  10

```

2)MULTI LEVEL PREEMPTIVE

```
#include <stdio.h>
```

```

struct Process {

    int pid;

    int at; // Arrival Time

    int bt; // Burst Time

    int rt; // Remaining Time

    int ct; // Completion Time

    int wt; // Waiting Time

    int tat; // Turnaround Time

    int type; // 0 = System, 1 = User

    int done; // 0 = not done, 1 = completed

};

```



```

int main() {
    int n, i, time = 0, completed = 0;

    printf("Enter total number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    // Input
    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i+1);
        p[i].pid = i + 1;

        printf("Enter Arrival Time: ");
        scanf("%d", &p[i].at);

        printf("Enter Burst Time: ");
        scanf("%d", &p[i].bt);

        printf("Enter Type (0 = System, 1 = User): ");
        scanf("%d", &p[i].type);

        p[i].rt = p[i].bt; // initialize remaining time
        p[i].done = 0;
    }

    // Preemptive Scheduling

```

```

while(completed < n) {

    int idx = -1;

    int best_at = 1e9;

    // Step 1: System processes (higher priority)
    for(i = 0; i < n; i++) {

        if(p[i].at <= time && p[i].done == 0 && p[i].type == 0 && p[i].rt > 0) {

            if(p[i].at < best_at) {

                best_at = p[i].at;

                idx = i;

            }

        }

    }

    // Step 2: User processes if no system process found
    if(idx == -1) {

        best_at = 1e9;

        for(i = 0; i < n; i++) {

            if(p[i].at <= time && p[i].done == 0 && p[i].type == 1 && p[i].rt > 0) {

                if(p[i].at < best_at) {

                    best_at = p[i].at;

                    idx = i;

                }

            }

        }

    }

    // Step 3: If no process available

```

```

if(idx == -1) {
    time++;
    continue;
}

// Execute for 1 time unit (PREEMPTIVE)
p[idx].rt--;
time++;

// If process completes
if(p[idx].rt == 0) {
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].done = 1;
    completed++;
}
}

// Output
float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tType\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%s\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt,
        (p[i].type == 0) ? "SYS" : "USR",

```

```

        p[i].ct, p[i].tat, p[i].wt);

    avg_wt += p[i].wt;

    avg_tat += p[i].tat;

}

printf("\nAverage WT = %.2f", avg_wt / n);

printf("\nAverage TAT = %.2f\n", avg_tat / n);

return 0;

}

```

RESULT:

```

PS D:\1bf24cs289> cd "d:\1bf24cs289\" ; if ($?) { gcc multiLevelPre.c -o multiLevelPre } ; if ($?) { .\multiLevelPre }
Enter total number of processes: 4

Process 1
Enter Arrival Time: 0
Enter Burst Time: 5
Enter Type (0 = System, 1 = User): 1

Process 2
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Type (0 = System, 1 = User): 0

Process 3
Enter Arrival Time: 2
Enter Burst Time: 4
Enter Type (0 = System, 1 = User): 1

Process 4
Enter Arrival Time: 3
Enter Burst Time: 2
Enter Type (0 = System, 1 = User): 0



| PID | AT | BT | Type | CT | TAT | WT |
|-----|----|----|------|----|-----|----|
| 1   | 0  | 5  | USR  | 10 | 10  | 5  |
| 2   | 1  | 3  | SYS  | 4  | 3   | 0  |
| 3   | 2  | 4  | USR  | 14 | 12  | 8  |
| 4   | 3  | 2  | SYS  | 6  | 3   | 1  |



Average WT = 3.50
Average TAT = 7.00

```

LAB PROGRAM 3

QUESTION:

Write a C program to simulate Real-Time CPU Scheduling

algorithms: (Any one)

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

CODE:

a) RATE- MONOTONIC:

```
// rate monotonic scheduling
```

```
#include<stdio.h>
```

```
#include<math.h>
```

```
struct tasks {
```

```
    int pid;
```

```
    int Ci;
```

```
    int Ti;
```

```
    int remaining;
```

```
    int next_arrival;
```

```
};
```

```
//gcd function for getting lcm
```

```
int gcd(int a, int b) {
```

```
    while(b != 0) {
```

```
        int temp = b;
```

```
        b = a % b;
```

```
        a = temp;
```

```
    }
```

```
    return a;
}
```

```
//lcm function for 2 numbers
```

```
int lcm(int a,int b) {
    return (a*b)/gcd(a,b);
}
```

```
//getting lcm of all tasks
```

```
int tasks_lcm(struct tasks t[],int n) {
    int result=t[0].Ti;

    for(int i=1;i<n;i++) {
        result=lcm(result,t[i].Ti);
    }

    return result;
}
```

```
int main() {
```

```
    int n;
    printf("enter number of tasks: ");
    scanf("%d",&n);
```

```
    struct tasks t[n];
```

```
    // input
```

```
    for(int i=0;i<n;i++) {
```

```
printf("Task %d\n", i+1);
```

```
printf("enter Ci: ");
```

```
scanf("%d", &t[i].Ci);
```

```
printf("enter Ti: ");
```

```
scanf("%d", &t[i].Ti);
```

```
t[i].pid = i+1;
```

```
// init for step 5
```

```
t[i].remaining = 0;
```

```
t[i].next_arrival = 0;
```

```
}
```

```
// Step 1: CPU utilization
```

```
float u=0;
```

```
for(int i=0;i<n;i++) {
```

```
    u += (float)t[i].Ci / t[i].Ti;
```

```
}
```

```
printf("CPU utilization: %f\n", u);
```

```
// Step 2: RMS bound
```

```
float bound = n * (pow(2, (float)1/n) - 1);
```

```
printf("Bound: %f\n", bound);
```

```
// Step 3
```

```
if(u <= bound) {
```

```
    printf("Schedulable using RMS\n");
} else {
    printf("Not guaranteed schedulable\n");
}
```

```
// Step 4: sort by Ti
for(int i=0;i<n;i++) {
    for(int j=i+1;j<n;j++) {
```

```
        if(t[i].Ti > t[j].Ti) {
            struct tasks temp = t[i];
            t[i] = t[j];
            t[j] = temp;
        }
    }
}
```

```
// Step 5: Scheduling simulation
```

```
int LIMIT = tasks_lcm(t,n); // simulate till lcm time
```

```
printf("\n--- Scheduling ---\n");
```

```
for(int time = 0; time < LIMIT; time++) {
```

```
    // check arrivals
```

```
    for(int i=0;i<n;i++) {
        if(time == t[i].next_arrival) {
            t[i].remaining = t[i].Ci;
```



```

        t[i].next_arrival += t[i].Ti;
    }
}

// pick highest priority task (smallest Ti)
int idx = -1;
for(int i=0;i<n;i++) {
    if(t[i].remaining > 0) {
        idx = i;
        break;
    }
}

// execute
if(idx != -1) {
    printf("Time %d: Task %d\n", time, t[idx].pid);
    t[idx].remaining--;
} else {
    printf("Time %d: Idle\n", time);
}
}

return 0;
}

```

RESULT:

```
PS D:\1bf24cs289> cd "d:\1bf24cs289\" ; if ($?) { gcc rateMono.c -o rateMono } ; if ($?) { .\rateMono }
enter number of tasks: 3
Task 1
enter Ci:
3
enter Ti: 20
Task 2
enter Ci: 2
enter Ti: 10
Task 3
enter Ci: 1
enter Ti: 5
CPU utilization: 0.550000
Bound: 0.779763
Schedulable using RMS

--- Scheduling ---
Time 0: Task 3
Time 1: Task 2
Time 2: Task 2
Time 3: Task 1
Time 4: Task 1
Time 5: Task 3
Time 6: Task 1
Time 7: Idle
Time 8: Idle
Time 8: Idle
Time 8: Idle
Time 9: Idle
Time 10: Task 3
Time 11: Task 2
Time 12: Task 2
Time 13: Idle
Time 14: Idle
Time 15: Task 3
Time 16: Idle
Time 17: Idle
Time 18: Idle
Time 19: Idle
```

b) EARLIEST-DEADLINE FIRST

```
#include <stdio.h>
```

```
struct process {
```

```
    int pid;
```

```
    int Ci;
```

```
    int Ti;
```

```
    int Di; // deadline = Ti - 1
```

```
    int remaining;
```

```
    int next_arrival;
```

```
    int absolute_deadline;
```

```
};
```

```
// GCD
```

```
int gcd(int a, int b) {
```

```
    while(b != 0) {
```

```
        int temp = b;
```

```
        b = a % b;
```

```
        a = temp;
```

```
    }
```

```
    return a;
```

```
}
```

```
// LCM
```

```
int lcm(int a, int b) {
```

```
    return (a * b) / gcd(a, b);
```

```
}
```

```

// LCM of all Ti

int find_lcm(struct process p[], int n) {
    int result = p[0].Ti;
    for(int i = 1; i < n; i++) {
        result = lcm(result, p[i].Ti);
    }
    return result;
}

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct process p[n];

    // input
    for(int i = 0; i < n; i++) {
        printf("Process %d\n", i+1);

        printf("Enter Ci: ");
        scanf("%d", &p[i].Ci);

        printf("Enter Ti: ");
        scanf("%d", &p[i].Ti);

        p[i].pid = i + 1;
    }
}

```

```

// given condition
p[i].Di = p[i].Ti - 1;

// init
p[i].remaining = 0;
p[i].next_arrival = 0;
p[i].absolute_deadline = p[i].Di;
}

// utilization (EDF uses Di)
float U = 0;
for(int i = 0; i < n; i++) {
    U += (float)p[i].Ci / p[i].Di;
}

printf("Utilization: %f\n", U);

if(U <= 1)
    printf("Feasible\n");
else
    printf("Not guaranteed feasible\n");

int LIMIT = find_lcm(p, n);

printf("\n--- EDF Scheduling (till %d) ---\n", LIMIT);

// scheduling loop

```

```

for(int time = 0; time < LIMIT; time++) {

    // arrivals
    for(int i = 0; i < n; i++) {
        if(time == p[i].next_arrival) {
            p[i].remaining = p[i].Ci;
            p[i].absolute_deadline = time + p[i].Di;
            p[i].next_arrival += p[i].Ti;
        }
    }

    // select earliest deadline
    int idx = -1;
    int min_deadline = 999999;

    for(int i = 0; i < n; i++) {
        if(p[i].remaining > 0) {
            if(p[i].absolute_deadline < min_deadline) {
                min_deadline = p[i].absolute_deadline;
                idx = i;
            }
        }
    }

    // execute
    if(idx != -1) {
        printf("Time %d: P%d\n", time, p[idx].pid);
        p[idx].remaining--;
    }
}

```

```

    } else {

        printf("Time %d: Idle\n", time);

    }

}

return 0;

}

```

RESULT:

```

PS D:\1bf24cs289> cd "d:\1bf24cs289\" ; if ($?) { gcc EDF.c -o EDF } ; if ($?) { .\EDF }
Enter number of processes: 2
Process 1
Enter Ci: 1
Enter Ti: 4
Process 2
Enter Ci: 2
Enter Ti: 5
Utilization: 0.833333
Feasible

--- EDF Scheduling (till 20) ---
Time 0: P1
Time 1: P2
Time 2: P2
Time 3: Idle
Time 4: P1
Time 5: P2
Time 6: P2
Time 7: Idle
Time 8: P1
Time 9: Idle
Time 10: P2
Time 11: P2
Time 12: P1
Time 13: Idle
Time 14: Idle
Time 15: P2
Time 16: P1
Time 17: P2
Time 18: Idle
Time 19: Idle
PS D:\1bf24cs289>

```

c)PROPORTIONAL SCHEDULING:

```
#include <stdio.h>
```

```
#define TIME_QUANTUM 10 // you can change this
```

```
struct process {
```

```
    int pid;
```

```
    int Ci;    // total execution time
```

```
    int remaining;
```

```
    int weight;
```

```
    int completed;
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct process p[n];
```

```
    int total_weight = 0;
```

```
    // input
```

```
    for(int i = 0; i < n; i++) {
```

```
        printf("Process %d\n", i+1);
```

```
        printf("Enter Ci: ");
```

```
        scanf("%d", &p[i].Ci);
```



```

printf("Enter weight: ");
scanf("%d", &p[i].weight);

p[i].pid = i + 1;
p[i].remaining = p[i].Ci;
p[i].completed = 0;

total_weight += p[i].weight;
}

int time = 0;
int completed_count = 0;

printf("\n--- Proportional Share Scheduling ---\n");

// Step 4
while(completed_count < n) {

    // Step 5: loop through all processes
    for(int i = 0; i < n; i++) {

        if(p[i].completed)
            continue;

        // Step 6: ready check (all ready here)

        // Step 8: calculate time slice

```

```

int time_slice = (p[i].weight * TIME_QUANTUM) / total_weight;

if(time_slice == 0) time_slice = 1; // avoid 0 slice

// Step 9: run process
printf("Time %d: P%d runs for %d units\n", time, p[i].pid, time_slice);

// Step 10
p[i].remaining -= time_slice;
time += time_slice;

// Step 11–12
if(p[i].remaining <= 0) {
    printf("P%d completed at time %d\n", p[i].pid, time);

    p[i].completed = 1;
    completed_count++;

// Step 13
    total_weight -= p[i].weight;
}
}
}

return 0;
}

```

RESULT:

```
PS D:\1bf24cs289> cd "d:\1bf24cs289\" ; if ($?) { gcc PropSch.c -o PropSch } ; if ($?) { .\PropSch }
● Enter number of processes: 3
Process 1
Enter Ci: 10
Enter weight: 1
Process 2
Enter Ci: 10
Enter weight: 2
Process 3
Enter Ci: 10
Enter weight: 3

--- Proportional Share Scheduling ---
Time 0: P1 runs for 1 units
Time 1: P2 runs for 3 units
Time 4: P3 runs for 5 units
Time 9: P1 runs for 1 units
Time 10: P2 runs for 3 units
Time 13: P3 runs for 5 units
P3 completed at time 18
Time 18: P1 runs for 3 units
Time 21: P2 runs for 6 units
P2 completed at time 27
Time 27: P1 runs for 10 units
P1 completed at time 37
○ PS D:\1bf24cs289> 
```

LAB PROGRAM 4

QUESTION:

Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

CODE:

a) Producer-Consumer problem using semaphores:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>


#define BUFFER_SIZE 5


int buffer[BUFFER_SIZE];

int in = 0, out = 0;


sem_t empty, full;

pthread_mutex_t mutex;


// Producer function

void* producer(void* arg) {

    int item;


    for(int i = 0; i < 10; i++) {

        item = rand() % 100;
```

```

sem_wait(&empty);    // wait if buffer is full

pthread_mutex_lock(&mutex); // enter critical section


buffer[in] = item;

printf("Produced: %d at index %d\n", item, in);

in = (in + 1) % BUFFER_SIZE;


pthread_mutex_unlock(&mutex); // exit critical section

sem_post(&full);    // increase filled slots


sleep(1);
}

return NULL;
}


// Consumer function
void* consumer(void* arg) {

    int item;


    for(int i = 0; i < 10; i++) {

        sem_wait(&full);    // wait if buffer is empty

        pthread_mutex_lock(&mutex);


        item = buffer[out];

        printf("Consumed: %d from index %d\n", item, out);

        out = (out + 1) % BUFFER_SIZE;
    }
}

```

```

    pthread_mutex_unlock(&mutex);

    sem_post(&empty);    // increase empty slots

    sleep(2);
}
return NULL;
}

int main() {
    pthread_t p, c;

    // Initialize semaphores
    sem_init(&empty, 0, BUFFER_SIZE); // initially all slots empty
    sem_init(&full, 0, 0);    // initially no items

    pthread_mutex_init(&mutex, NULL);

    // Create threads
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    // Wait for threads to finish
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    // Cleanup
    sem_destroy(&empty);
    sem_destroy(&full);
}

```

```
pthread_mutex_destroy(&mutex);
```

```
return 0;
```

```
}
```

RESULT:

```
PS D:\1bf24cs289\ada> cd "d:\1bf24cs289\ada\" ; if ($?) { gcc producer_consumer.c -o producer_consumer } ; if ($?) { .\producer_consumer }
Produced: 41 at index 0
Consumed: 41 from index 0
Produced: 67 at index 1
Consumed: 67 from index 1
Produced: 34 at index 2
Produced: 0 at index 3
Consumed: 34 from index 2
Produced: 69 at index 4
Produced: 24 at index 0
Consumed: 0 from index 3
Produced: 78 at index 1
Produced: 58 at index 2
Consumed: 69 from index 4
Produced: 62 at index 3
Produced: 64 at index 4
Consumed: 24 from index 0
Consumed: 78 from index 1
Consumed: 58 from index 2
Consumed: 62 from index 3
Consumed: 64 from index 4
```

b) Dining-Philosopher's problem:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>


#define N 5 // Number of philosophers


pthread_mutex_t forks[N];


void* philosopher(void* num) {
    int id = *(int*)num;

    while (1) {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        // Deadlock prevention strategy
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[(id + 1) % N]); // right fork
            pthread_mutex_lock(&forks[id]);           // left fork
        } else {
            pthread_mutex_lock(&forks[id]);           // left fork
            pthread_mutex_lock(&forks[(id + 1) % N]); // right fork
        }

        printf("Philosopher %d is eating\n", id);
        sleep(2);
    }
}
```



```

        pthread_mutex_unlock(&forks[id]);
        pthread_mutex_unlock(&forks[(id + 1) % N]);

        printf("Philosopher %d finished eating\n", id);
    }
}

int main() {
    pthread_t threads[N];
    int ids[N];

    // Initialize mutexes (forks)
    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }

    // Create philosopher threads
    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    // Join threads (never actually ends)
    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }
}

```

```
    return 0;
}
```

RESULT:

```
PS D:\1bf24cs289\ada> cd "d:\1bf24cs289\ada\" ; if ($?) { gcc dining_philosopher.c -o dining_philosopher } ; if ($?) { .\dining_philosopher }
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 1 is eating
Philosopher 3 is eating
Philosopher 4 is eating
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 2 is eating
Philosopher 2 finished eating
Philosopher 3 is eating
Philosopher 0 is eating
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 2 is thinking
Philosopher 1 is eating
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 4 is eating
Philosopher 4 finished eating
Philosopher 0 is eating
Philosopher 2 is eating
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 4 is thinking
Philosopher 3 is eating
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 1 is eating
Philosopher 1 finished eating
Philosopher 2 is eating
Philosopher 4 is eating
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 1 is thinking
```

LAB PROGRAM 5

QUESTION:

Write a C program to simulate:

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

CODE:

a) Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resource types: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], need[n][m];
```

```
    int avail[m];
```

```
    // Allocation Matrix
```

```
    printf("\nEnter Allocation Matrix:\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        for(int j = 0; j < m; j++) {
```

```
            scanf("%d", &alloc[i][j]);
```

```
        }
```

```
}
```

```
// Max Matrix
```

```
printf("\nEnter Max Matrix:\n");
```

```
for(int i = 0; i < n; i++) {
```

```
    for(int j = 0; j < m; j++) {
```

```
        scanf("%d", &max[i][j]);
```

```
    }
```

```
}
```

```
// Available Resources
```

```
printf("\nEnter Available Resources:\n");
```

```
for(int i = 0; i < m; i++) {
```

```
    scanf("%d", &avail[i]);
```

```
}
```

```
// Calculate Need Matrix
```

```
for(int i = 0; i < n; i++) {
```

```
    for(int j = 0; j < m; j++) {
```

```
        need[i][j] = max[i][j] - alloc[i][j];
```

```
    }
```

```
}
```

```
printf("\nNeed Matrix:\n");
```

```
for(int i = 0; i < n; i++) {
```

```
    for(int j = 0; j < m; j++) {
```

```
        printf("%d ", need[i][j]);
```

```
    }
```

```
    printf("\n");  
}
```

```
int finish[n], safeSeq[n];  
int work[m];
```

```
for(int i = 0; i < n; i++)  
    finish[i] = 0;
```

```
for(int i = 0; i < m; i++)  
    work[i] = avail[i];
```

```
int count = 0;
```

```
while(count < n) {  
    int found = 0;
```

```
    for(int i = 0; i < n; i++) {
```

```
        if(finish[i] == 0) {
```

```
            int possible = 1;
```

```
            for(int j = 0; j < m; j++) {  
                if(need[i][j] > work[j]) {  
                    possible = 0;  
                    break;  
                }  
            }
```

```
}
```

```
if(possible) {
```

```
    for(int j = 0; j < m; j++) {
```

```
        work[j] += alloc[i][j];
```

```
    }
```

```
    safeSeq[count] = i;
```

```
    count++;
```

```
    finish[i] = 1;
```

```
    found = 1;
```

```
}
```

```
}
```

```
}
```

```
if(found == 0) {
```

```
    break;
```

```
}
```

```
}
```

```
if(count == n) {
```

```
    printf("\nSystem is in SAFE state\n");
```

```
    printf("Safe Sequence: ");
```

```
    for(int i = 0; i < n; i++) {
```

```
        printf("P%d ", safeSeq[i]);
```

```
    }
```

```

        printf("\n");
    }

    else {

        printf("\nSystem is NOT in safe state\n");

    }

    return 0;

}

```

RESULT:

```

PS D:\1bf24cs289\os> cd "d:\1bf24cs289\os\" ; if ($?) { gcc bankers_algo.c -o bankers_algo } ; if ($?) { .\bankers_algo }
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE state
Safe Sequence: P1 P3 P4 P0 P2

```

b) Deadlock Detection:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resource types: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], request[n][m];
```

```
    int avail[m];
```

```
    // Allocation Matrix
```

```
    printf("\nEnter Allocation Matrix:\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        for(int j = 0; j < m; j++) {
```

```
            scanf("%d", &alloc[i][j]);
```

```
        }
```

```
    }
```

```
    // Request Matrix
```

```
    printf("\nEnter Request Matrix:\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        for(int j = 0; j < m; j++) {
```



```

        scanf("%d", &request[i][j]);
    }
}

// Available Resources
printf("\nEnter Available Resources:\n");
for(int i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}

int finish[n];
int work[m];

// Initialize work = available
for(int i = 0; i < m; i++) {
    work[i] = avail[i];
}

// Initialize finish[]
for(int i = 0; i < n; i++) {

    int empty = 1;

    for(int j = 0; j < m; j++) {
        if(alloc[i][j] != 0) {
            empty = 0;
            break;
        }
    }
}

```

```
}
```

```
if(empty)
```

```
    finish[i] = 1;
```

```
else
```

```
    finish[i] = 0;
```

```
}
```

```
while(1) {
```

```
    int found = 0;
```

```
    for(int i = 0; i < n; i++) {
```

```
        if(finish[i] == 0) {
```

```
            int possible = 1;
```

```
            for(int j = 0; j < m; j++) {
```

```
                if(request[i][j] > work[j]) {
```

```
                    possible = 0;
```

```
                    break;
```

```
                }
```

```
            }
```

```
            if(possible) {
```

```

        for(int j = 0; j < m; j++) {
            work[j] += alloc[i][j];
        }

        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
    break;
}

int deadlock = 0;

for(int i = 0; i < n; i++) {

    if(finish[i] == 0) {
        deadlock = 1;
        break;
    }
}

if(deadlock) {

    printf("\nDeadlock detected among processes:\n");

```

```

for(int i = 0; i < n; i++) {

    if(finish[i] == 0) {

        printf("P%d ", i);

    }

}

printf("\n");

}

else {

    printf("\nNo deadlock detected\n");

}

return 0;

}

```

RESULT:

```

PS D:\1bf24cs289\os> cd "d:\1bf24cs289\os\" ; if ($?) { gcc deadlock_detection.c -o deadlock_detection } ; if ($?) { .\deadlock_detection }
Enter number of processes: 3
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3

Enter Request Matrix:
0 0 0
2 0 2
0 0 1

Enter Available Resources:
0 0 0

Deadlock detected among processes:
P1 P2
PS D:\1bf24cs289\os>

```

LAB PROGRAM 6

QUESTION:

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit b) Best-fit c) First-fit

CODE:

```
#include <stdio.h>
```

```
void firstFit(int blockSize[], int blocks, int processSize[], int processes) {
```

```
    int allocation[10];
```

```
    for (int i = 0; i < processes; i++)
```

```
        allocation[i] = -1;
```

```
    for (int i = 0; i < processes; i++) {
```

```
        for (int j = 0; j < blocks; j++) {
```

```
            if (blockSize[j] >= processSize[i]) {
```

```
                allocation[i] = j;
```

```
                blockSize[j] -= processSize[i];
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("\n--- First Fit ---\n");
```

```
    printf("Process No.\tProcess Size\tBlock No.\n");
```

```
    for (int i = 0; i < processes; i++) {
```

```

printf("%d\t\t%d\t\t", i + 1, processSize[i]);

if (allocation[i] != -1)
    printf("%d\n", allocation[i] + 1);
else
    printf("Not Allocated\n");
}
}

void bestFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[10];

    for (int i = 0; i < processes; i++)
        allocation[i] = -1;

    for (int i = 0; i < processes; i++) {
        int bestIdx = -1;

        for (int j = 0; j < blocks; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
        }
    }
}

```

```

        blockSize[bestIdx] -= processSize[i];
    }
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (int i = 0; i < processes; i++) {
    printf("%d\t%d\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[10];

    for (int i = 0; i < processes; i++)
        allocation[i] = -1;

    for (int i = 0; i < processes; i++) {
        int worstIdx = -1;

        for (int j = 0; j < blocks; j++) {
            if (blockSize[j] >= processSize[i]) {

```

```

        if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
            worstIdx = j;
        }
    }
}

```

```

if (worstIdx != -1) {
    allocation[i] = worstIdx;
    blockSize[worstIdx] -= processSize[i];
}
}

```

```

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

```

```

for (int i = 0; i < processes; i++) {
    printf("%d\t%d\t", i + 1, processSize[i]);

```

```

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

```

```

int main() {
    int blocks, processes;

```



```
int blockSize[10], processSize[10];
```

```
int block1[10], block2[10], block3[10];
```

```
printf("Enter number of memory blocks: ");
```

```
scanf("%d", &blocks);
```

```
printf("Enter sizes of %d memory blocks:\n", blocks);
```

```
for (int i = 0; i < blocks; i++) {
```

```
    scanf("%d", &blockSize[i]);
```

```
    block1[i] = blockSize[i];
```

```
    block2[i] = blockSize[i];
```

```
    block3[i] = blockSize[i];
```

```
}
```

```
printf("\nEnter number of processes: ");
```

```
scanf("%d", &processes);
```

```
printf("Enter sizes of %d processes:\n", processes);
```

```
for (int i = 0; i < processes; i++)
```

```
    scanf("%d", &processSize[i]);
```

```
firstFit(block1, blocks, processSize, processes);
```

```
bestFit(block2, blocks, processSize, processes);
```

```
worstFit(block3, blocks, processSize, processes);
```

```
return 0;
```

```
}
```

RESULT:

```
PS D:\1bf24cs289\os> cd "d:\1bf24cs289\os\" ; if ($?) { gcc lp6.c -o lp6 } ; if ($?) { .\lp6 }
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600

Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.    Process Size    Block No.
1              212           2
2              417           5
3              112           2
4              426          Not Allocated

--- Best Fit ---
Process No.    Process Size    Block No.
1              212           4
2              417           2
3              112           3
4              426           5

--- Worst Fit ---
Process No.    Process Size    Block No.
1              212           5
2              417           2
3              112           5
4              426          Not Allocated
PS D:\1bf24cs289\os> 
```

LAB PROGRAM 7

QUESTION:

Write a C program to simulate page replacement algorithms.

a) FIFO b) LRU c) Optimal

CODE:

```
#include <stdio.h>
```

```
#define MAX 50
```

```
// Function to print frame contents
```

```
void printFrames(int frames[], int f) {
```

```
    for (int i = 0; i < f; i++) {
```

```
        if (frames[i] == -1)
```

```
            printf("- ");
```

```
        else
```

```
            printf("%d ", frames[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
// FIFO Algorithm
```

```
void FIFO(int pages[], int n, int f) {
```

```
    int frames[MAX];
```

```
    int pageFaults = 0;
```

```
    int index = 0;
```

```

for (int i = 0; i < f; i++)
    frames[i] = -1;

printf("FIFO Page Replacement Process:\n");

for (int i = 0; i < n; i++) {
    int found = 0;

    for (int j = 0; j < f; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }

    if (!found) {
        frames[index] = pages[i];
        index = (index + 1) % f;

        pageFaults++;

        printf("PF No. %d: ", pageFaults);
        printFrames(frames, f);
    }
}

printf("FIFO Page Faults: %d\n", pageFaults);
}

```

```

// LRU Algorithm

void LRU(int pages[], int n, int f) {
    int frames[MAX], time[MAX];

    int pageFaults = 0;

    int counter = 0;

    for (int i = 0; i < f; i++) {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\nLRU Page Replacement Process:\n");

    for (int i = 0; i < n; i++) {
        int found = 0;

        for (int j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }

        if (!found) {
            int pos = 0;

```

```

        for (int j = 1; j < f; j++) {
            if (time[j] < time[pos])
                pos = j;
        }

        frames[pos] = pages[i];

        counter++;
        time[pos] = counter;

        pageFaults++;

        printf("PF No. %d: ", pageFaults);
        printFrames(frames, f);
    }
}

printf("LRU Page Faults: %d\n", pageFaults);
}

// Function for Optimal Algorithm
int predict(int pages[], int frames[], int n, int index, int f) {
    int farthest = index;
    int pos = -1;

    for (int i = 0; i < f; i++) {
        int j;

```

```

    for (j = index; j < n; j++) {
        if (frames[i] == pages[j]) {
            if (j > farthest) {
                farthest = j;
                pos = i;
            }
            break;
        }
    }

    if (j == n)
        return i;
}

if (pos == -1)
    return 0;
else
    return pos;
}

// Optimal Algorithm
void Optimal(int pages[], int n, int f) {
    int frames[MAX];
    int pageFaults = 0;

    for (int i = 0; i < f; i++)
        frames[i] = -1;

```

```
printf("\nOptimal Page Replacement Process:\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    int found = 0;
```

```
    for (int j = 0; j < f; j++) {
```

```
        if (frames[j] == pages[i]) {
```

```
            found = 1;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (!found) {
```

```
        int empty = -1;
```

```
        for (int j = 0; j < f; j++) {
```

```
            if (frames[j] == -1) {
```

```
                empty = j;
```

```
                break;
```

```
            }
```

```
        }
```

```
        if (empty != -1) {
```

```
            frames[empty] = pages[i];
```

```
        } else {
```

```
            int pos = predict(pages, frames, n, i + 1, f);
```

```
            frames[pos] = pages[i];
```



```

    }

    pageFaults++;

    printf("PF No. %d: ", pageFaults);
    printFrames(frames, f);
}
}

printf("Optimal Page Faults: %d\n", pageFaults);
}

int main() {
    int f, n;
    int pages[MAX];

    printf("Enter the number of Frames: ");
    scanf("%d", &f);

    printf("Enter the length of reference string: ");
    scanf("%d", &n);

    printf("Enter the reference string: ");

    for (int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    FIFO(pages, n, f);

```

```
LRU(pages, n, f);
```

```
Optimal(pages, n, f);
```

```
return 0;
```

```
}
```

RESULT:

```
PS D:\1bf24cs289> cd "d:\1bf24cs289\os\" ; if ($?) { gcc lp7.c -o lp7 } ; if ($?) { .\lp7 }
Enter the number of Frames: 3
Enter the length of reference string: 6
Enter the reference string: 1 4 2 3 2 1
FIFO Page Replacement Process:
PF No. 1: 1 - -
PF No. 2: 1 4 -
PF No. 3: 1 4 2
PF No. 4: 3 4 2
PF No. 5: 3 1 2
FIFO Page Faults: 5

LRU Page Replacement Process:
PF No. 1: 1 - -
PF No. 2: 1 4 -
PF No. 3: 1 4 2
PF No. 4: 3 4 2
PF No. 5: 3 1 2
LRU Page Faults: 5

Optimal Page Replacement Process:
PF No. 1: 1 - -
PF No. 2: 1 4 -
PF No. 3: 1 4 2
PF No. 4: 1 3 2
Optimal Page Faults: 4
PS D:\1bf24cs289\os> 
```